Universidade Fernando Pessoa

Sistemas Operativos

Ficha de Trabalho 3

Objectivos:

- Usar as chamadas ao sistema `fork(), waitpid(), wait() e exit();`

1. Considere o programa fork.c disponível na plataforma:
   a. Descreva o funcionamento das chamadas ao sistema `fork()` e `exit()`.
   b. Execute este código. Qual o processo que escreve para a saída padrão em primeiro lugar? O pai ou o filho? Tem alguma garantia que seja sempre assim? Porquê?

2. Analise o código do ficheiro `forkwait.c` disponível na plataforma.
   a. Descreva o funcionamento das chamadas ao sistema `wait()` e `waitpid()`.
   b. Escreva um programa que lance 5 filhos. Cada um deles imprime o seu PID para o terminal e um valor aleatório entre 0 e 100. Antes de terminar deve devolver ao pai esse número. O pai deve esperar pela finalização ordenada dos seus filhos, recolher o respectivo valor enviado e imprimir o PID do seu filho e o valor recebido. Por fim o processo pai termina.

3. Crie um programa que possua um vector de inteiros (com o nome `primos`) com 10 posições contendo os primeiros 10 números primos. Esse programa deve lançar 2 filhos. Um deles vai somar as posições pares do vector (0,2,4,6,8) e outro vai somar as posições ímpares (1,3,5,7,9). Cada filho deve colocar a zero as posições que somar. Cada filho deve imprimir depois o seu PID, o valor da soma efectuada e o conteúdo do vector primos por fim termina enviando o valor da soma como código de finalização ao seu pai (usando a chamada `exit()`). O pai deve aguardar pela finalização dos seus filhos, recolher o valor da finalização de cada um, imprimir o PID do filho e o código de finalização e o conteúdo do vector `primos` e terminar. Como justifica os valores apresentados no vector `primos` pelos filhos e pelo pai? Haverá partilha de variáveis entre o Pai e os Filhos? De que forma?

Bibliografia:

[1] Kernigham B., Ritchie D. – *The C Programming Language* – Addison Wesley.

[2] Stevens R., Rago S.– *Advanced Programming in the UNIX environment 3rd ed.* – Addison Wesley, 2013